

JAVA ASSESSMENT CO1 AT3 PSEUDO CODE

1.

SIMATS Online Programming Lab

JAVA PROGRAMMING

Sivarangjenii B

192521375RC

WRITING

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
START
Class Student:
name,roll no ,marks ,average
PUBLIC:
METHOD acceptdetails()
INPUT name,rollno,marks
END METHOD
METHOD calculateaverage()
average and marks/3
END METHOD
METHOD displayresults()
DISPLAY name,rollno,average
IF average >= 50
DISPLAY "PASS"
ELSE
DISPLAY"FAIL"
END IF
END METHOD
END CLASS
CREATE Student object
CALL acceptdetails()
CALL calculateAverage()
CALL displayResults()
STOP
```

2.

SIMATS Online Programming Lab

JAVA PROGRAMMING

Sivarangjenii B
192521375RC

WRITING

← Back

Q2 2. Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
START
CLASS Book
PRIVATE:
title, author, ISBN, isAvailable
PUBLIC:
METHOD issueBook()
isAailable is FALSE
END METHOD
METHOD returnBook()
isAvailable is True
END METHOD
METHOD displayDetails()
DISPLAY title, author, ISBN, isAvailable
END METHOD
END CLASS
CREATE 3 Book objects
CALL issueBook()
CALL returnBook()
CALL displayDetails()
STOP
```

3.

SIMATS Online Programming Lab
JAVA PROGRAMMING

Sivarangjenii B
192521375RC

WRITING

← Back

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
START
Class Employees
empId, name, basicpay, designation
METHOD calculateGross()
gross, basicpay + HRA + DA
END METHOD
METHOD calculateNet()
net , gross - PF - Tax
END METHOD
METHOD displayPaySlip()
DISPLAY empId, name, gross, net
END METHOD
END CLASS
CREATE Employee object
CALL calculateGross()
CALL calculateNet()
CALL displayPaySlip()
STOP
```

4.

SIMATS Online Programming Lab

JAVA PROGRAMMING

 **Sivarangjenii B**
192521375RC

WRITING

← Back

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
START
CLASS shape
PUBLIC:
METHOD calculateArea()
END METHOD
END CLASS
CLASS Cricle EXTENDS Shape
METHOD calculateArea()
area <- 3.14 * r * r
END METHOD
CLASS Rectangle EXTENDS Shape
METHOD calculateArea()
area <- length* breadth
END METHOD
CLASS Triangle EXTENDS Shape
METHOD calculateArea()
area <- 0.5 * base * height
END METHOD
CREATE Cricle, Rectangle, Triangle objects
CALL calculateArea()
STOP
```

5.

SIMATS Online Programming Lab

JAVA PROGRAMMING

Sivarangjenii B
192521375RC

WRITING

← Back

Q5 Vehicle Registration System [BL3 – Apply]10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.
OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
START
CLASS Vehicle
PUBLIC:
  regNo, owner, fuelType
METHOD displayDetails()
END METHOD
END CLASS
CLASS twoWheeler EXTENDS Vehicle
METHOD displaydetails()
  DISPLAY regNo, owner, fuelType, enginecapacity
END METHOD
CLASS FourWheeler EXTENDS vehicle
METHOD displayDetails()
  DISPLAY regNo, owner, fuelType, seatingCapacity, Acstatus
END METHOD
CREATE TwoWheeler, FourWheeler object
CALL displayDetails()
STOP
```

6.

SIMATS Online Programming Lab

JAVA PROGRAMMING

 **Sivarangjenii B**
192521375RC

E WRITING

← Back

Q6 Bank Account Hierarchy [BL4 – Analyze]10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
START
CLASS BankAccount
PUBLIC:
accNo, balance, holder
METHOD withdraw()
END METHOD
END CLASS
CLASS SavingAccont EXTENDS bankAccount
METHOD withdraw()
balance <- balance - amount
END METHOD
CLASS CurrentAccount EXTENDS bankAccount
METHOD withdraw()
balance <- balance - amount + overdraft
END METHOD
CREATE Savingccount, CurrentAccount objects
CALL withdraw()
STOP
```

7.

WRITING

← Back

Q7 Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
START
CLASS Patient
PRIVATE:
diagnosis, prescription
PUBLIC :
patientId, name
METHOD generateBill()
END METHOD
END CLASS
CLASS InPatient EXTENDS patient
METHOD generateBill()
END METHOD()
END CLASS
CLASS InPatient EXTENDS patient
METHOD generateBill()
bill <- dailyCharges
END METHOD
CLASS OutPatient EXTENDS patient
METHOD generateBill()
bill <- consultationfree
END METHOD
CREATE Inpatient, OutPatient object
CALL generateBill()
STOP
```


8.

SIMATS Online Programming Lab

JAVA PROGRAMMING

 **Sivarangjenii B**
192521375RC

WRITING

← Back

Q8 E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
START
CLASS Product
PUBLIC:
    productId, name, price
    METHOD applyDiscount()
END METHOD
END CLASS
CLASS Electronic EXTENDS Product
    METHOD applyDiscount()
    price <- price - discount
END METHOD
CLASS Clothing EXTENDS Product
    METHOD applyDiscount()
    price <- price - discount
END METHOD
CLASS Grocery EXTENDS Product
    METHOD applyDiscount()
    price <- price - discount
END METHOD
CREATE Electronics , Clothing , Grocery object
CALL applyDiscount()
STOP
```

9.

SIMATS Online Programming Lab
JAVA PROGRAMMING
Sivarangjenii B
192521375RC

E WRITING
Back

Q9 University Staff Hierarchy [BL4 – Analyze] 10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```

START
CLASS staff
PRIVATE id
PROTECTED salary
METHOD calculateAllowance()
RETURN salary*5%
END METHOD
END CLASS
CLASS TeacherStaff INHERITS Staffs
METHOD calculateAllowance()
RETURN salary*10%
END METHOD
END CLASS
CLASS lecture INHERITS Teachingstaff
METHOD calculateAllowance()
RETURN salary*15%
END METHOD
END CLASS
CLASS NonTecachingStaff INHERITS Staff
METHOD calculateAllowance()
RETURN salary*8%
END METHOD
END CLASS
STOP
  
```

10.

E WRITING

← Back

Q10 Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

START

CLASS SmartDevice

METHOD executeCommand(command)

DISPLAY "Execute command"

END METHOD

END CLASS

CLASS SmartLight INHERITS SmartDevice

METHOD executeCommand(command)

DISPLAY "Light:" + command

END METHOD

END CLASS

CLASS SmartThermostat INHERITS SmartDevice

METHOD executeCommand(command)

DISPLAY"Thermost:" + command

END METHOD

END CLASS

CLASS SmartLock INHERITS SmartDevice

PRIVATE pin

METHOD executeCommand(command)

DISPLAY "Lock:" + command

END METHOD

END CLASS

FOR EACH device IN devices

device executeCommand("ON")

END FOR

STOP